



TRAININGCRED
INSTITUTE



Training Programme

Skills for the Modern Workforce

+254759509615 | www.trainingcred.com

TRANSACT-SQL AND PYTHON PROGRAMMING TRAINING TRAINING PROGRAM

Venue: Cape Town

Duration: 10 Days | Format: Simulation-Based & Experiential

PROGRAM OVERVIEW

This 10-day programme equips Central Bank of Lesotho data professionals with practical, measurable proficiency in Transact-SQL and Python programming as complementary tools for enterprise database management, analytical automation, and performance-optimised reporting. Delivered in Cape Town, the programme moves participants from foundational environment configuration through advanced query design, stored procedure engineering, Python ETL development, statistical analysis, automated reporting, API integration, performance monitoring, security hardening, and strategic deployment planning — producing tangible workplace artefacts at every stage. The curriculum is calibrated to foundation-to-intermediate level, meaning participants are expected to arrive with basic SQL familiarity and will leave capable of independently designing, implementing, and optimising integrated T-SQL and Python solutions in a regulated financial environment. Advanced topics such as machine learning and time series analysis are taught at conceptual-awareness and operational-application depth, not production-engineering depth, which is consistent with the programme's stated level and duration. Every day's content is grounded in the Central Bank's operational context — foreign exchange data, regulatory reporting, audit trail requirements, and high-availability system obligations — ensuring that skills transfer directly to the participant's desk the following week.

Target Audience

- Database Administrators responsible for SQL Server query optimisation, performance tuning, and schema management
- Data Analysts who extract, transform, and report on financial and operational data
- Business Intelligence Developers building automated reports and analytical dashboards
- IT Systems Officers supporting database infrastructure and application integration
- Financial Reporting Officers who consume and validate data outputs from core banking systems
- Data Engineers responsible for ETL pipeline design and maintenance
- Compliance and Audit Officers who require technical literacy to assess database controls and audit trails
- ICT Project Managers overseeing database and automation initiatives within the Central Bank

Prerequisites

- Working familiarity with basic SQL SELECT statements, WHERE clauses, and simple JOINS — no advanced T-SQL required
- Basic understanding of relational database concepts (tables, primary keys, foreign keys, indexes)
- No prior Python programming experience required; participants with Python basics will progress faster through Day 1 afternoon content
- Pre-reading: Microsoft documentation on SQL Server Architecture Overview (provided two weeks before Day 1)
- Laptop with administrator rights for software installation; IT department to pre-approve SQL Server ODBC driver installation before the programme starts

Learning Materials & Resources

- Comprehensive participant workbook covering all 10 modules with annotated code examples, exercise instructions, and reference tables
- Domain-specific toolkit including T-SQL optimisation pattern library, Python ETL script templates, Jinja2 report templates, and security assessment checklist
- Online resource library with 6-month access to curated Microsoft Docs, Python documentation, and supplementary video walkthroughs

- Pre-loaded SQL Server training database with financial transactions, foreign exchange, and regulatory datasets for hands-on exercises
- Named tool access: SQL Server Management Studio 20.x, VS Code with Python extension, Jupyter Notebook environment, and Git repository scaffold
- Post-programme action planning workbook with 30-60-90 day milestone templates and KPI tracking sheets

ASSESSMENT & CERTIFICATION

Participants must achieve 80% attendance for certification. Assessment components are weighted as follows:

Assessment Component	Weight
Daily Reflective Journals & Peer Feedback	10%
Simulation/Exercise Performance & Debrief Quality	30%
Domain-Specific Analytical Exercise	20%
Capstone Project Development & Presentation	30%
Participation & Engagement	10%
Total	100%

TRAINING METHODOLOGY

The allocation reflects the programme's stated outcome-driven, practical methodology: 40% hands-on labs and guided practice ensures participants build genuine coding proficiency through repeated, scaffolded execution rather than passive observation; 20% simulation and integration exercises (including the two-phase progressive simulation) develops the ability to apply skills under realistic time and stakeholder pressure; the remaining 40% is distributed across lectures, peer review, case studies, and reflection to provide conceptual grounding, collaborative validation, and structured transfer of learning — consistent with a 70-20-10 experiential weighting adapted for a technical programming course.

Methodology	Allocation
Hands On Labs And Guided Practice	40%
Simulation And Integration Exercises	20%
Interactive Lectures And Demonstrations	15%
Collaborative Troubleshooting And Peer Review	12%
Case Study Analysis	8%
Reflection And Action Planning	5%
Total	100%

PROGRAM ARCHITECTURE

Day	Theme	Modules	Signature Activity
-----	-------	---------	--------------------

Day 1	FOUNDATIONS — ARCHITECTURE, ENVIRONMENT, AND PROFESSIONAL SETUP	Module 1: Database Programming Fundamentals and Environment Setup	HANDS-ON LAB: Integrated Environment Configuration and Live Python-to-SQL Server Connection
Day 2	ADVANCED T-SQL — JOINS, CTEs, WINDOW FUNCTIONS, AND EXECUTION PLANS	Module 2: Advanced T-SQL Query Design and Optimization	DATA LAB: Query Optimisation Benchmarking Lab — Before-and-After Execution Plan Analysis
Day 3	STORED PROCEDURES, FUNCTIONS, TRIGGERS, AND DATABASE AUTOMATION	Module 3: Stored Procedures, Functions, and Database Automation	HANDS-ON LAB: Automated Data Validation Stored Procedure with Error Handling and Audit Logging
Day 4	PYTHON DATABASE CONNECTIVITY, PANDAS, NUMPY, AND ETL WORKFLOWS	Module 4: Python Data Manipulation and Database Connectivity	HANDS-ON LAB: Python ETL Pipeline — Extract, Transform, and Load Financial Data to SQL Server
Day 5	STATISTICAL ANALYSIS, VISUALIZATION, AND MIDPOINT INTEGRATION SIMULATION	Module 5: Statistical Analysis and Data Visualization with Python	SIMULATION: Phase 1 Simulation — Central Bank Analytical Reporting Scenario
Day 6	AUTOMATED REPORTING, WORKFLOW INTEGRATION, AND PHASE 2 SIMULATION	Module 6: Automated Reporting and Workflow Integration	SIMULATION: Phase 2 Simulation — Automated Monthly Reporting System Under Operational Stress
Day 7	API INTEGRATION, EXTERNAL DATA SOURCES, AND DATA QUALITY ASSURANCE	Module 7: API Integration and External Data Sources	WORKSHOP: End-to-End API Integration — External Data Extraction, Quality Validation, and SQL Server Load
Day 8	PERFORMANCE MONITORING, SYSTEM OPTIMIZATION, AND PRODUCTION READINESS	Module 8: Performance Monitoring and System Optimization	DATA LAB: Comprehensive Monitoring Dashboard — Database Performance and Python Workflow Observability
Day 9	SECURITY, COMPLIANCE, BEST PRACTICES, AND ENTERPRISE DEPLOYMENT PLANNING	Module 9: Security, Compliance, and Best Practices, Module 10: Enterprise Deployment and Strategic Planning	HANDS-ON LAB: Security Assessment and Hardening — Stored Procedures and Python Scripts
Day 10	STRATEGIC ROADMAPS, CAPSTONE PRESENTATIONS, AND PROGRAMME CLOSE	Module 10: Enterprise Deployment and Strategic Planning	SIMULATION: Capstone Executive Panel Presentations — Enterprise Database Programming Initiative

DETAILED DAILY SCHEDULE

DAY 1: FOUNDATIONS — ARCHITECTURE, ENVIRONMENT, AND PROFESSIONAL SETUP

HANDS-ON LAB: Integrated Environment Configuration and Live Python-to-SQL Server Connection — Participants individually configure SQL Server Management Studio, a Python IDE (VS Code with the Python extension), and the pyodbc connection library, then execute a verified live query from Python against a pre-provisioned SQL Server instance. Each participant documents their connection string, resolves any driver or firewall issues encountered, and commits their project scaffold to a shared Git repository — establishing the baseline environment used throughout the remaining nine days.

Day 1 establishes the technical and professional foundation upon which all subsequent programming work will rest. Participants configure their integrated development environments, connect Python to SQL Server, and internalize the architectural principles that govern efficient database programming. This day orients the Central Bank of Lesotho cohort to the standards of enterprise-grade database development.

Modules: Module 1: Database Programming Fundamentals and Environment Setup

Time	Module	Session / Activity	Method
09:00 - 09:30		Registration and Welcome	Administrative
09:30 - 10:30	Module 1: Database Programming Fundamentals and Environment Setup	T-SQL architecture, query execution principles, and optimization fundamentals in enterprise database environments	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 1: Database Programming Fundamentals and Environment Setup	Python programming basics, data types, control structures, and best practices for database integration projects	Interactive Lecture
11:45 - 13:00	Module 1: Database Programming Fundamentals and Environment Setup	Development environment setup including SQL Server Management Studio, Python IDEs, and connection libraries configuration	Hands-on Lab
13:00 - 14:00			Break
14:00 - 15:00	Module 1: Database Programming Fundamentals and Environment Setup	Version control, documentation standards, and project structure for maintainable database programming solutions	Workshop
15:00 - 15:30	Module 1: Database Programming Fundamentals and Environment Setup	★ Exercise: Configure integrated development environment and establish connection between Python and SQL Server database	Hands-on Lab
15:30 - 15:45			Break
15:45 - 16:30	Module 1: Database Programming Fundamentals and Environment Setup	Facilitated debrief: reviewing connection results, troubleshooting environment issues across the cohort, and aligning on project standards	Facilitated Reflection
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 1 Learning Outcomes

1. Map the T-SQL query execution pipeline and identify how architectural decisions affect performance in enterprise database environments
2. Classify Python data types and control structures according to their appropriate use cases in database integration projects
3. Compare SQL Server Management Studio and Python IDE configurations to select the optimal toolchain for a given development task
4. Apply version control and documentation standards to organise a maintainable database programming project structure
5. Interpret connection library configuration outputs to diagnose and resolve environment setup issues
6. Execute the integrated environment configuration exercise to establish a verified live connection between Python and SQL Server

FACILITATOR NOTES

- *Setup: Pre-provision a shared SQL Server 2019 (or SQL Server 2022) instance accessible from the training venue network. Prepare a Git repository (GitHub or Azure DevOps) with a starter project scaffold. Ensure each participant workstation has VS Code, SQL Server Management Studio 20.x, Python 3.11+, pyodbc, and SQLAlchemy pre-installed or available via an offline installer package.*
- *Technology: Test all ODBC driver connections (Microsoft ODBC Driver 18 for SQL Server) from the venue network before Day 1 starts. Prepare an offline installer USB for participants whose machines encounter download restrictions. Have a facilitator laptop with a working connection as a live reference.*
- *Groups: Keep participants in their natural job-role groupings for the morning (DBAs together, analysts together) to surface role-specific environment questions, then mix groups for the afternoon workshop to encourage cross-functional knowledge sharing.*
- *Energy: The post-lunch version control workshop should be run as a hands-on activity with Git command-line practice — avoid lecture-only delivery after lunch. Use pair programming to keep energy high.*
- *Accessibility: Provide printed quick-reference cards for Git commands and ODBC connection string syntax. For participants with visual impairments, ensure VS Code accessibility settings (high-contrast theme, screen reader mode) are pre-configured.*
- *Prep: Distribute Day 2 pre-reading — the Microsoft documentation page on SQL Server query execution plans — and ask participants to note two questions they want answered. Confirm all participants can access the shared SQL Server instance before they leave.*

Additional Notes:

Day 1 Notes

DAY 2: ADVANCED T-SQL — JOINS, CTEs, WINDOW FUNCTIONS, AND EXECUTION PLANS

DATA LAB: Query Optimisation Benchmarking Lab — Before-and-After Execution Plan Analysis — Participants receive a set of deliberately poorly written T-SQL queries against a pre-loaded financial transactions dataset. Using SQL Server Management Studio's actual execution plan viewer and SET STATISTICS TIME/IO, they identify the top performance bottlenecks, apply targeted index and query rewrites, and record quantified before-and-after metrics (logical reads, CPU time, elapsed time). Each participant produces a one-page optimisation findings document that is exchanged during the peer review session.

Day 2 moves participants into the analytical power of advanced T-SQL, equipping them to write sophisticated queries that are both functionally correct and performance-conscious. By examining execution plans and applying index strategies, participants begin to think like database engineers rather than mere query writers. This analytical mindset is essential for supporting the Central Bank's data-intensive reporting requirements.

Modules: Module 2: Advanced T-SQL Query Design and Optimization

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 2: Advanced T-SQL Query Design and Optimization	Day 2 orientation: recap of Day 1 environment setup, introduction to advanced T-SQL design principles and the day's performance benchmarking approach	Facilitated Reflection
09:30 - 10:30	Module 2: Advanced T-SQL Query Design and Optimization	Complex JOIN operations, subqueries, CTEs, and window functions for sophisticated data analysis and reporting	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 2: Advanced T-SQL Query Design and Optimization	Query execution plans, index utilization strategies, and performance tuning methodologies for large-scale databases	Demonstration
11:45 - 13:00	Module 2: Advanced T-SQL Query Design and Optimization	Advanced data manipulation using MERGE statements, table expressions, and bulk operations for efficient data processing	Guided Practice
13:00 - 14:00			Break
14:00 - 15:00	Module 2: Advanced T-SQL Query Design and Optimization	Error handling, transaction management, and concurrency control for robust database operations	Case Study Analysis
15:00 - 15:30	Module 2: Advanced T-SQL Query Design and Optimization	★ Exercise: Optimize poorly performing queries and document measurable performance improvements with execution plan analysis	Data Lab
15:30 - 15:45			Break
15:45 - 16:30	Module 2: Advanced T-SQL Query Design and Optimization	Peer review of query optimisation outputs: participants exchange execution plan analyses, validate improvement claims, and consolidate findings	Peer Review
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 2 Learning Outcomes

1. Apply complex JOIN operations, CTEs, and window functions to solve multi-layered data analysis and reporting problems
2. Interpret query execution plans to identify index utilisation gaps and performance bottlenecks in large-scale databases
3. Analyse MERGE statements and bulk operation patterns to evaluate their suitability for high-volume data processing scenarios
4. Compare error handling and transaction management strategies to select the most robust approach for concurrent database operations
5. Evaluate poorly performing queries against execution plan evidence to prioritise optimisation interventions
6. Execute the query optimisation exercise and document measurable before-and-after performance improvements with quantified metrics

FACILITATOR NOTES

- *Setup:* Pre-load the training SQL Server instance with a financial transactions dataset of sufficient size (minimum 500,000 rows) to produce meaningful execution plan differences. Prepare five deliberately unoptimised query scripts at varying difficulty levels. Ensure Query Store is enabled on the training database.
- *Technology:* Confirm that SQL Server Management Studio's execution plan viewer is functional on all participant machines. Prepare a fallback set of pre-captured execution plan screenshots in case of connectivity issues. Have SQL Server Profiler available as a secondary monitoring tool.
- *Groups:* For the Data Lab exercise, participants work individually to ensure each person builds personal proficiency. For the peer review, pair participants with different job roles (DBA with analyst) to surface complementary perspectives on the same execution plan.
- *Energy:* The post-lunch case study on error handling and transaction management should use a real-world financial system failure scenario to maintain engagement. Ask participants to identify parallels with systems they manage at the Central Bank.
- *Accessibility:* Provide execution plan diagrams in both graphical and XML text formats so participants who prefer text-based analysis can work in their preferred mode. Allow extra time for participants less familiar with SSMS graphical tools.
- *Prep:* Distribute the Day 3 pre-reading on stored procedure design patterns from Microsoft Docs. Ask participants to bring one example of a repetitive database task from their work that could benefit from automation.

Additional Notes:

Day 2 Notes

DAY 3: STORED PROCEDURES, FUNCTIONS, TRIGGERS, AND DATABASE AUTOMATION

HANDS-ON LAB: Automated Data Validation Stored Procedure with Error Handling and Audit Logging — Participants build a T-SQL stored procedure that accepts a table name and validation rule set as parameters, executes data quality checks (null checks, referential integrity, range validation), logs all violations to an audit table with timestamps and severity levels, and returns a structured result set. The procedure must implement TRY/CATCH error handling, use parameterised dynamic SQL to prevent injection, and write to a dedicated ErrorLog table. The output is reviewed in the group walkthrough session.

Day 3 elevates participants from query writing to database engineering by introducing reusable, automated database objects. Stored procedures, user-defined functions, and triggers form the backbone of enterprise-grade automation, and participants build these with security-conscious dynamic SQL practices. This capability directly supports the Central Bank's need for reliable, auditable automated database operations.

Modules: Module 3: Stored Procedures, Functions, and Database Automation

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 3: Stored Procedures, Functions, and Database Automation	Day 3 orientation: connecting automation concepts to the Central Bank's operational context and reviewing Day 2 optimisation benchmarks	Facilitated Reflection
09:30 - 10:30	Module 3: Stored Procedures, Functions, and Database Automation	Stored procedure design patterns, parameter handling, and return value strategies for flexible, reusable database operations	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 3: Stored Procedures, Functions, and Database Automation	User-defined functions, table-valued functions, and scalar functions for modular data processing and business logic implementation	Guided Practice
11:45 - 13:00	Module 3: Stored Procedures, Functions, and Database Automation	Triggers for data validation, audit trails, and automated business rule enforcement across database operations	Hands-on Lab
13:00 - 14:00			Break
14:00 - 15:00	Module 3: Stored Procedures, Functions, and Database Automation	Dynamic SQL generation, SQL injection prevention, and secure coding practices for robust database applications	Workshop
15:00 - 15:30	Module 3: Stored Procedures, Functions, and Database Automation	★ Exercise: Develop automated data validation stored procedure with comprehensive error handling and logging capabilities	Hands-on Lab
15:30 - 15:45			Break
15:45 - 16:30			Group Exercise
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 3 Learning Outcomes

1. Apply stored procedure design patterns and parameter handling strategies to build flexible, reusable database operation modules
2. Classify user-defined functions — scalar versus table-valued — according to their appropriate role in modular data processing logic
3. Analyse trigger behaviour across insert, update, and delete events to evaluate their suitability for audit trail and rule enforcement scenarios
4. Compare dynamic SQL generation approaches against SQL injection prevention requirements to select secure implementation patterns
5. Interpret execution logs from a stored procedure to diagnose error handling gaps and implement corrective logging capabilities
6. Develop an automated data validation stored procedure with comprehensive error handling and logging that meets enterprise quality standards

FACILITATOR NOTES

- *Setup:* Pre-create an audit schema and ErrorLog table on the training database. Prepare three sample datasets with known data quality issues (nulls, out-of-range values, orphaned foreign keys) for participants to validate against. Provide a starter stored procedure template with the signature and logging table structure already defined.
- *Technology:* Ensure all participants have CREATE PROCEDURE and CREATE TABLE permissions on their assigned schemas. Prepare a reference implementation of the completed stored procedure for the facilitator to use during the group walkthrough debrief.
- *Groups:* Form groups of three for the group walkthrough — mix experience levels so that participants with prior T-SQL automation experience can articulate design rationale to less experienced peers. Each group should review at least two other groups' procedures.
- *Energy:* The post-lunch dynamic SQL workshop should be run as a live coding challenge — present a vulnerable stored procedure on screen and ask participants to identify and fix injection vulnerabilities in real time before the facilitator reveals the solution.
- *Accessibility:* Provide a printed T-SQL error handling reference card (TRY/CATCH syntax, RAISERROR vs THROW comparison). For participants who complete the exercise early, provide an extension challenge: add a retry mechanism for deadlock scenarios.
- *Prep:* Distribute Day 4 pre-reading on pyodbc and SQLAlchemy connection patterns. Ask participants to identify one repetitive data extraction task in their current role that could be automated with a Python script.

Additional Notes:

Day 3 Notes

DAY 4: PYTHON DATABASE CONNECTIVITY, PANDAS, NUMPY, AND ETL WORKFLOWS

HANDS-ON LAB: Python ETL Pipeline — Extract, Transform, and Load Financial Data to SQL Server — Participants write a Python script using pyodbc or SQLAlchemy to extract a financial transactions dataset from SQL Server into a pandas DataFrame, apply a defined set of transformations (currency normalisation, date parsing, null imputation, aggregation), validate the output using NumPy statistical checks, and load the transformed results back to a staging table in SQL Server. The script must include connection error handling and a row-count reconciliation check. The completed script is exchanged with a peer for the collaborative troubleshooting session.

Day 4 bridges the T-SQL world with Python's data manipulation ecosystem, giving participants the skills to extract, transform, and load data programmatically. Working with pyodbc, SQLAlchemy, pandas, and NumPy, participants build the ETL pipelines that automate repetitive data workflows. For the Central Bank of Lesotho, this capability translates directly into faster, more reliable data preparation for analytical and reporting processes.

Modules: Module 4: Python Data Manipulation and Database Connectivity

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 4: Python Data Manipulation and Database Connectivity	Day 4 orientation: framing the Python-to-SQL Server connectivity challenge and mapping today's ETL work to real Central Bank data pipeline scenarios	Facilitated Reflection
09:30 - 10:30	Module 4: Python Data Manipulation and Database Connectivity	Python database connectivity using pyodbc, SQLAlchemy, and pandas for efficient data extraction and manipulation	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 4: Python Data Manipulation and Database Connectivity	Pandas DataFrame operations, data cleaning techniques, and transformation workflows for analytical data preparation	Hands-on Lab
11:45 - 13:00	Module 4: Python Data Manipulation and Database Connectivity	NumPy arrays, mathematical operations, and statistical calculations for quantitative analysis and data validation	Guided Practice
13:00 - 14:00			Break
14:00 - 15:00	Module 4: Python Data Manipulation and Database Connectivity	File I/O operations, CSV/Excel processing, and data format conversions for integrated data workflows	Data Lab
15:00 - 15:30	Module 4: Python Data Manipulation and Database Connectivity	★ Exercise: Create Python script that extracts data from SQL Server, performs transformations, and loads results back to database	Hands-on Lab
15:30 - 15:45			Break
15:45 - 16:30	Module 4: Python Data Manipulation and Database Connectivity	Collaborative troubleshooting: pairs diagnose each other's ETL scripts, identify transformation errors, and validate final database load results	Peer Review
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 4 Learning Outcomes

1. Apply Python database connectivity libraries — pyodbc, SQLAlchemy, and pandas — to extract and load data from a live SQL Server instance
2. Execute pandas DataFrame transformation workflows to clean and reshape raw financial datasets into analysis-ready structures
3. Analyse NumPy array operations and statistical calculations to validate the mathematical integrity of transformed data
4. Interpret file I/O outputs across CSV and Excel formats to diagnose data format conversion errors in integrated workflows
5. Compare connection management strategies in pyodbc versus SQLAlchemy to evaluate performance and reliability trade-offs
6. Develop a Python ETL script that extracts SQL Server data, performs documented transformations, and loads verified results back to the database

FACILITATOR NOTES

- *Setup: Pre-load a raw financial transactions table with intentional data quality issues (mixed date formats, null amounts, inconsistent currency codes) for participants to clean. Prepare a target staging table schema for the load step. Ensure pandas 2.x, NumPy 1.26+, pyodbc, and SQLAlchemy 2.x are installed on all participant machines.*
- *Technology: Test openpyxl (for Excel I/O) and xlrd availability on all machines. Prepare a fallback CSV version of the dataset in case Excel library issues arise. Have a working reference ETL script available for the facilitator to demonstrate key steps if participants get stuck.*
- *Groups: Pair participants for the collaborative troubleshooting session based on complementary strengths — pair a participant strong in Python with one stronger in SQL to encourage mutual learning. Pairs should swap scripts and run them against the same database to validate results.*
- *Energy: The post-lunch Data Lab on file I/O should involve participants processing a real-format CSV export (mimicking a Central Bank report extract) to maintain contextual relevance and engagement.*
- *Accessibility: Provide a pandas cheat sheet covering the most commonly used DataFrame methods. For participants with limited Python experience, provide a partially completed ETL script template with the connection and load sections pre-written so they can focus on the transformation logic.*
- *Prep: Distribute Day 5 pre-reading on SciPy statistical functions and Matplotlib chart types. Inform participants that Day 5 includes the Phase 1 Simulation and ask them to review their Day 1–4 work in preparation.*

Additional Notes:

Day 4 Notes

DAY 5: STATISTICAL ANALYSIS, VISUALIZATION, AND MIDPOINT INTEGRATION SIMULATION

SIMULATION: Phase 1 Simulation — Central Bank Analytical Reporting Scenario — Teams of four receive a simulated Central Bank scenario: the Governor's office requires an urgent analytical report on foreign exchange reserve movements over the past 24 months, including trend analysis, anomaly detection, and a 3-month forecast. Teams must write T-SQL queries to extract the relevant data, use pandas and SciPy for statistical analysis, apply Scikit-learn for pattern detection, and produce a Matplotlib/Seaborn visualisation dashboard. Teams have 45 minutes to produce and present their report. The facilitator benchmarks outputs against defined accuracy, completeness, and visualisation quality criteria.

Day 5 marks the pivotal midpoint of the programme, where analytical capability meets visual communication. Participants apply SciPy, Matplotlib, Seaborn, Scikit-learn, and time series techniques to transform raw data into compelling, insight-rich reports. The Phase 1 Simulation challenges teams to integrate everything learned so far — T-SQL extraction, Python transformation, and statistical visualisation — in a realistic Central Bank analytical scenario.

Modules: Module 5: Statistical Analysis and Data Visualization with Python

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 5: Statistical Analysis and Data Visualization with Python	Day 5 orientation: midpoint programme review, framing the Phase 1 Simulation challenge, and introducing today's analytical reporting objectives	Facilitated Reflection
09:30 - 10:30	Module 5: Statistical Analysis and Data Visualization with Python	Statistical analysis using SciPy, descriptive statistics, hypothesis testing, and correlation analysis for data insights	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 5: Statistical Analysis and Data Visualization with Python	Data visualization with Matplotlib and Seaborn for creating professional charts, graphs, and analytical dashboards	Hands-on Lab
11:45 - 13:00	Module 5: Statistical Analysis and Data Visualization with Python	Machine learning fundamentals using Scikit-learn for predictive modeling and pattern recognition in business data	Guided Practice
13:00 - 14:00			Break
14:00 - 14:45	Module 5: Statistical Analysis and Data Visualization with Python	Time series analysis techniques for forecasting, trend analysis, and seasonal pattern identification	Data Lab
14:45 - 15:30	Module 5: Statistical Analysis and Data Visualization with Python	★ Exercise: Develop comprehensive analytical report combining T-SQL data extraction with Python statistical analysis and visualization	Simulation & Debrief
15:30 - 15:45			Break
15:45 - 16:30			Group Exercise
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 5 Learning Outcomes

1. Apply SciPy statistical functions to execute descriptive statistics, hypothesis testing, and correlation analysis on financial datasets
2. Interpret Matplotlib and Seaborn chart outputs to evaluate whether visualisations accurately represent underlying data distributions and trends
3. Analyse Scikit-learn model outputs to classify patterns in business data and assess the predictive reliability of the results
4. Apply time series decomposition techniques to identify trend, seasonal, and residual components in financial time data
5. Develop a comprehensive analytical report that integrates T-SQL data extraction with Python statistical analysis and professional visualisation
6. Evaluate the end-to-end integrated pipeline produced in the Phase 1 Simulation against defined accuracy and performance benchmarks

FACILITATOR NOTES

- *Setup:* Pre-load a 24-month foreign exchange reserve time series dataset into the training SQL Server database. Prepare a one-page scenario brief for each team describing the Governor's reporting requirements and the acceptance criteria for the output. Ensure SciPy 1.11+, Scikit-learn 1.3+, Matplotlib 3.7+, and Seaborn 0.12+ are installed.
- *Technology:* Prepare Jupyter Notebook starter files with import statements and data connection templates pre-written so teams can focus on analysis rather than boilerplate. Have a fallback static dataset in CSV format in case database connectivity issues arise during the simulation.
- *Groups:* Form cross-functional teams of four for the simulation, mixing DBAs, analysts, and developers. Assign team roles: Data Extractor (T-SQL), Transformation Lead (pandas), Statistics Lead (SciPy/Scikit-learn), and Visualisation Lead (Matplotlib/Seaborn). Rotate roles from Day 4 pairs.
- *Energy:* The simulation runs post-lunch and is inherently high-energy due to its time-constrained, team-based format. Brief teams clearly on the 45-minute limit and use a visible countdown timer. The debrief should be structured as a mini-presentation (5 minutes per team) to maintain momentum.
- *Accessibility:* Ensure Jupyter Notebooks are accessible via browser so participants who prefer not to use a local IDE can work in JupyterLab. Provide a printed statistical reference card covering key SciPy functions and their parameters.
- *Prep:* Distribute the Day 6 Phase 2 Simulation brief so teams can begin thinking about the automated reporting challenge. Ask participants to note one stakeholder communication pain point from their current reporting workflows.

Additional Notes:

Day 5 Notes

DAY 6: AUTOMATED REPORTING, WORKFLOW INTEGRATION, AND PHASE 2 SIMULATION

SIMULATION: Phase 2 Simulation — Automated Monthly Reporting System Under Operational Stress — Teams build on their Phase 1 analytical pipeline to create a fully automated monthly reporting system. The base scenario requires a scheduled Python workflow that extracts data via T-SQL stored procedures, processes it with pandas, renders a Jinja2-formatted HTML report, and distributes it via email to a defined recipient list. At 15-minute intervals, the facilitator introduces escalating challenges: (1) the scheduled job fails silently and must be diagnosed via logs; (2) a senior stakeholder requests an additional chart with 10 minutes' notice; (3) a data privacy concern is raised about a field included in the distributed report; (4) the SMTP server is unavailable and an alternative distribution method must be implemented. Teams must adapt their pipeline in real time and demonstrate a working automated run at the debrief.

Day 6 transitions participants into the implementation phase, where they design and build end-to-end automated reporting pipelines that deliver outputs to stakeholders without manual intervention. Task scheduling, email automation, Jinja2 templating, and deployment configuration are combined in the Phase 2 Simulation, where teams construct and stress-test a realistic monthly reporting system aligned to Central Bank operational requirements.

Modules: Module 6: Automated Reporting and Workflow Integration

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 6: Automated Reporting and Workflow Integration	Day 6 orientation: connecting Day 5 analytical outputs to automated distribution, introducing the Phase 2 Simulation challenge brief	Facilitated Reflection
09:30 - 10:30	Module 6: Automated Reporting and Workflow Integration	Task scheduling using Windows Task Scheduler, cron jobs, and Python scheduling libraries for automated workflow execution	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 6: Automated Reporting and Workflow Integration	Email automation, report distribution, and notification systems for stakeholder communication and alert management	Hands-on Lab
11:45 - 13:00	Module 6: Automated Reporting and Workflow Integration	Template-based reporting using Jinja2, automated document generation, and professional output formatting	Guided Practice
13:00 - 14:00			Break
14:00 - 14:45	Module 6: Automated Reporting and Workflow Integration	Configuration management, environment variables, and deployment strategies for production automation systems	Workshop
14:45 - 15:30	Module 6: Automated Reporting and Workflow Integration	★ Exercise: Build automated monthly reporting system that extracts data via T-SQL, processes with Python, and distributes formatted reports	Simulation & Debrief
15:30 - 15:45			Break
15:45 - 16:30			Group Exercise
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 6 Learning Outcomes

1. Design a task scheduling architecture using Windows Task Scheduler and Python scheduling libraries to automate recurring workflow execution
2. Implement email automation and notification logic to distribute formatted analytical reports to defined stakeholder groups
3. Develop Jinja2-templated report documents that meet professional formatting standards for Central Bank executive audiences
4. Construct configuration management and environment variable structures that support reliable deployment across development and production systems
5. Execute the automated monthly reporting system exercise by integrating T-SQL extraction, Python processing, and formatted report distribution
6. Evaluate the Phase 2 Simulation pipeline against scheduling reliability, output quality, and stakeholder communication effectiveness benchmarks

FACILITATOR NOTES

- *Setup:* Pre-configure a local SMTP test server (using Python's smtplib module or a tool such as Mailhog) on the training network so email automation can be tested without live internet access. Prepare Jinja2 HTML report templates at two complexity levels. Ensure the APScheduler and schedule Python libraries are installed.
- *Technology:* Prepare a Windows Task Scheduler XML task template and a cron expression reference card. Have a backup distribution method (writing the report to a shared network folder) ready for the SMTP failure escalation in the Phase 2 Simulation. Test all scheduling mechanisms the evening before Day 6.
- *Groups:* Retain the same cross-functional teams from the Phase 1 Simulation to allow teams to build on their established working relationships and existing pipeline code. Assign a new 'Deployment Lead' role for Day 6 to ensure configuration management responsibilities are distributed.
- *Energy:* The Phase 2 Simulation is the highest-energy activity of the programme. Brief teams clearly on the escalation format before the simulation begins so they are not surprised by the interruptions. Use a visible timer and announce each escalation with a brief written card delivered to each team.
- *Accessibility:* Provide a printed Jinja2 template syntax reference. For participants less familiar with email protocols, provide a pre-written SMTP connection code block they can adapt rather than write from scratch.
- *Prep:* Distribute Day 7 pre-reading on RESTful API authentication patterns (OAuth 2.0, API keys). Ask participants to identify one external data source (market data, regulatory feed, or public API) relevant to their Central Bank role.

Additional Notes:

Day 6 Notes

DAY 7: API INTEGRATION, EXTERNAL DATA SOURCES, AND DATA QUALITY ASSURANCE

WORKSHOP: End-to-End API Integration — External Data Extraction, Quality Validation, and SQL Server Load — Participants build a Python integration solution that authenticates against a pre-configured test REST API (serving simulated exchange rate or economic indicator data), handles pagination and rate limiting, parses the JSON response, maps fields to a target SQL Server table schema, applies a defined set of data quality acceptance criteria (completeness, range, format), and loads only validated records. Rejected records are written to a quarantine table with rejection reasons. The completed solution is exchanged with another team for cross-team integration testing in the peer review session.

Day 7 extends the Central Bank cohort's data acquisition capability beyond internal databases to the broader data ecosystem. Participants build API consumers, parse JSON and XML payloads, apply web scraping techniques, and implement rigorous data quality validation before loading external data into SQL Server. These skills are critical for integrating market data, regulatory feeds, and third-party financial data sources.

Modules: Module 7: API Integration and External Data Sources

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 7: API Integration and External Data Sources	Day 7 orientation: mapping external data source integration to Central Bank use cases — market data, regulatory APIs, and inter-bank feeds	Facilitated Reflection

Time	Module	Session / Activity	Method
09:30 - 10:30	Module 7: API Integration and External Data Sources	RESTful API consumption using Python requests library, authentication handling, and rate limiting strategies	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 7: API Integration and External Data Sources	JSON and XML data processing, parsing techniques, and integration with relational database structures	Hands-on Lab
11:45 - 13:00	Module 7: API Integration and External Data Sources	Web scraping fundamentals using BeautifulSoup and Scrapy for automated data collection from web sources	Guided Practice
13:00 - 14:00			Break
14:00 - 15:00	Module 7: API Integration and External Data Sources	Data source validation, quality assessment, and integration testing for reliable external data incorporation	Data Lab
15:00 - 15:30	Module 7: API Integration and External Data Sources	★ Exercise: Create integration solution that pulls data from external API, validates quality, and stores in SQL Server database	Workshop
15:30 - 15:45			Break
15:45 - 16:30			Peer Review
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 7 Learning Outcomes

1. Implement RESTful API consumption scripts using the Python requests library with authentication handling and rate limiting controls
2. Develop JSON and XML parsing routines that accurately map external data structures to relational database schemas
3. Apply BeautifulSoup web scraping techniques to extract structured data from authorised web sources for database integration
4. Design data source validation and quality assessment procedures that enforce acceptance criteria before database insertion
5. Construct integration testing suites that verify the reliability of external data incorporation workflows end-to-end
6. Execute the API integration exercise to deliver a validated, quality-assured external data load into a SQL Server database

FACILITATOR NOTES

- *Setup:* Deploy a lightweight mock REST API server (using Flask or FastAPI on the facilitator laptop) that serves simulated exchange rate data with intentional quality issues (missing fields, out-of-range values, malformed dates). Prepare target and quarantine table schemas on the training SQL Server instance. Ensure requests 2.31+, BeautifulSoup4, and Scrapy are installed.
- *Technology:* Ensure the mock API server is accessible from all participant machines on the training network. Prepare a Postman collection (or Bruno, the open-source alternative) with pre-configured API requests for participants to inspect the API structure before coding. Have a static JSON file as a fallback if the mock server has issues.
- *Groups:* For the cross-team integration testing review, assign teams to test a solution built by a team they have not previously paired with. Provide a structured testing checklist covering: authentication, pagination handling, quality validation logic, and database load verification.
- *Energy:* The post-lunch Data Lab on data source validation should use a competitive format — teams race to identify the most data quality issues in a provided dataset within 20 minutes — to maintain afternoon energy levels.
- *Accessibility:* Provide a printed JSON/XML parsing reference card covering Python's json module and xml.etree.ElementTree. For participants less familiar with web scraping, provide a pre-built BeautifulSoup scraper template targeting a static HTML page so they can focus on the parsing logic.
- *Prep:* Distribute Day 8 pre-reading on SQL Server DMVs and Query Store. Ask participants to think about one production system they manage where performance monitoring is currently reactive rather than proactive.

Additional Notes:

Day 7 Notes

DAY 8: PERFORMANCE MONITORING, SYSTEM OPTIMIZATION, AND PRODUCTION READINESS

DATA LAB: Comprehensive Monitoring Dashboard — Database Performance and Python Workflow Observability — Participants design and partially implement a monitoring dashboard that combines SQL Server DMV queries (top resource-consuming queries, blocking chains, index fragmentation) with Python workflow execution metrics (script run times, row counts processed, error rates) captured via Python's logging module. The dashboard output is a structured HTML or CSV report generated by a Python script that queries both the SQL Server DMVs and a Python execution log table. Participants define alerting thresholds for each metric and document the rationale. The design is presented and stress-tested in the group exercise debrief.

Day 8 equips participants to move beyond building solutions to sustaining and improving them in production. Using DMVs, Query Store, Extended Events, and Python profiling tools, participants establish performance baselines, design monitoring dashboards, and implement logging frameworks that keep enterprise systems healthy and observable. This operational expertise is essential for the Central Bank's expectation of high-availability, auditable database systems.

Modules: Module 8: Performance Monitoring and System Optimization

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 8: Performance Monitoring and System Optimization	Day 8 orientation: framing production monitoring as a strategic obligation for Central Bank data systems and introducing today's observability design challenge	Facilitated Reflection
09:30 - 10:30	Module 8: Performance Monitoring and System Optimization	Database performance monitoring using DMVs, Query Store, and Extended Events for proactive system management	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 8: Performance Monitoring and System Optimization	Python application profiling, memory management, and optimization techniques for efficient script execution	Hands-on Lab
11:45 - 13:00	Module 8: Performance Monitoring and System Optimization	Logging frameworks, error tracking, and debugging strategies for maintainable production systems	Guided Practice
13:00 - 14:00			Break
14:00 - 15:00	Module 8: Performance Monitoring and System Optimization	Capacity planning, scalability assessment, and performance baseline establishment for growing data volumes	Case Study Analysis
15:00 - 15:30	Module 8: Performance Monitoring and System Optimization	★ Exercise: Design comprehensive monitoring dashboard that tracks both database performance and Python workflow execution metrics	Data Lab
15:30 - 15:45			Break
15:45 - 16:30			Group Exercise
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 8 Learning Outcomes

1. Demonstrate the use of DMVs, Query Store, and Extended Events to proactively identify and diagnose database performance issues
2. Implement Python application profiling and memory management techniques to optimise script execution efficiency in production workflows
3. Construct logging frameworks and error tracking systems that support maintainable, debuggable production database applications
4. Formulate capacity planning and scalability assessments using established performance baselines and projected data volume growth
5. Design a comprehensive monitoring dashboard that tracks database performance metrics and Python workflow execution status simultaneously
6. Assess the monitoring dashboard produced in the lab exercise against defined alerting thresholds and operational observability standards

FACILITATOR NOTES

- *Setup:* Pre-configure a set of DMV queries covering `sys.dm_exec_query_stats`, `sys.dm_exec_requests`, and `sys.dm_db_index_usage_stats` on the training SQL Server instance. Prepare a Python execution log table schema. Ensure Python's `cProfile` module and `memory_profiler` library are installed for the profiling lab.
- *Technology:* Prepare a reference set of Extended Events session definitions for common monitoring scenarios (long-running queries, deadlocks). Have SQL Server Query Store reports pre-configured on the training database. Ensure participants have `VIEW SERVER STATE` permission on the SQL Server instance.
- *Groups:* For the dashboard design exercise, allow participants to work in pairs to manage the scope of the deliverable within the 30-minute time slot. For the group debrief, each pair presents their alerting threshold rationale to the full group for collective critique.
- *Energy:* The post-lunch capacity planning case study should reference a real-world scenario of a financial institution managing database growth — use publicly available information about database scaling challenges in African banking contexts to maintain relevance.
- *Accessibility:* Provide a printed DMV quick-reference card listing the most commonly used dynamic management views and their key columns. For participants less familiar with Python profiling, provide a pre-written `cProfile` wrapper function they can apply to their existing scripts.
- *Prep:* Distribute Day 9 pre-reading on SQL Server role-based access control and the OWASP SQL Injection Prevention Cheat Sheet. Ask participants to bring one example of a security or compliance requirement from their Central Bank role to the Day 9 morning discussion.

Additional Notes:

Day 8 Notes

DAY 9: SECURITY, COMPLIANCE, BEST PRACTICES, AND ENTERPRISE DEPLOYMENT PLANNING

HANDS-ON LAB: Security Assessment and Hardening — Stored Procedures and Python Scripts — Participants receive a set of intentionally vulnerable stored procedures and Python database scripts containing SQL injection vulnerabilities, hardcoded credentials, excessive database permissions, and missing audit logging. Using a structured security assessment checklist aligned to OWASP and SQL Server security best practices, participants identify each vulnerability, document its risk level, implement the recommended fix, and verify the remediation. The hardened code is committed to the shared Git repository with a security assessment report documenting findings and changes made.

Day 9 addresses the non-negotiable dimensions of enterprise database programming — security, regulatory compliance, and disciplined deployment. Participants conduct a security assessment of real procedures and scripts, implement recommended hardening measures, and then pivot to deployment planning, change management, and ROI definition. For the Central Bank of Lesotho, operating in a regulated financial environment, these skills are as critical as any technical capability.

Modules: Module 9: Security, Compliance, and Best Practices | Module 10: Enterprise Deployment and Strategic Planning

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 9: Security, Compliance, and Best Practices	Day 9 orientation: security and compliance as strategic imperatives in a central banking environment — framing today's assessment and deployment planning work	Facilitated Reflection
09:30 - 10:30	Module 9: Security, Compliance, and Best Practices	Database security principles, role-based access control, and SQL injection prevention in stored procedures and dynamic queries	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:15	Module 9: Security, Compliance, and Best Practices	Python security considerations, credential management, and secure coding practices for database applications	Guided Practice
11:15 - 12:00	Module 9: Security, Compliance, and Best Practices	Data privacy regulations, audit trail implementation, and compliance documentation for regulated industries	Case Study Analysis
12:00 - 13:00	Module 9: Security, Compliance, and Best Practices	★ Exercise: Conduct security assessment of existing database procedures and Python scripts, implementing recommended security enhancements	Hands-on Lab
13:00 - 14:00			Break
14:00 - 14:45	Module 9: Security, Compliance, and Best Practices	Code review processes, testing strategies, and quality assurance procedures for reliable database programming solutions	Workshop
14:45 - 15:30	Module 10: Enterprise Deployment and Strategic Planning	Deployment planning, environment management, and release procedures for database and Python application changes	Group Exercise
15:30 - 15:45			Break
15:45 - 16:30	Module 10: Enterprise Deployment and Strategic Planning	Change management strategies, user training programs, and adoption planning for new automated systems	Case Study Analysis
16:30 - 17:00		Reflection, Journaling & Day Briefing	Facilitated Reflection

Day 9 Learning Outcomes

1. Implement role-based access control and SQL injection prevention measures across stored procedures and dynamic queries to meet financial sector security standards
2. Assess Python credential management and secure coding practices against a defined threat model for database-connected applications
3. Formulate audit trail and compliance documentation structures that satisfy data privacy regulations applicable to Central Bank operations
4. Design a code review and quality assurance process that enforces security and reliability standards across the development lifecycle
5. Construct a deployment plan and change management strategy for rolling out database and Python automation changes across production environments
6. Evaluate performance KPIs, success metrics, and ROI calculation frameworks to demonstrate measurable return on database programming investments

FACILITATOR NOTES

- *Setup:* Prepare a set of five intentionally vulnerable stored procedures and three Python scripts with documented security flaws. Create a security assessment checklist template covering injection prevention, credential management, permission scoping, and audit logging. Ensure participants have a test schema where they can safely execute and test their hardened code.
- *Technology:* Prepare a reference implementation of each hardened script for the facilitator to use during debrief. Ensure Python's `python-dotenv` library is installed for the credential management exercise. Have the OWASP SQL Injection Prevention Cheat Sheet available as a printed reference.
- *Groups:* The security assessment lab runs individually to ensure each participant builds personal security assessment skills. The code review workshop (14:00) should be run in pairs using a structured peer review checklist to simulate a real code review process.
- *Energy:* The post-lunch code review workshop should be run as a structured pair activity with clear time-boxed review rounds (15 minutes per review) to maintain focus and energy. Use a physical checklist card that participants mark off as they review.
- *Accessibility:* Provide the security assessment checklist in both digital and printed formats. For participants less familiar with security concepts, provide a glossary of key terms (SQL injection, RBAC, least privilege, credential vault) before the morning lecture.
- *Prep:* Distribute the Day 10 capstone presentation brief and assessment rubric. Ask participants to spend 30 minutes in the evening drafting the outline of their capstone deployment plan, focusing on the specific database programming initiative they will propose for the Central Bank.

Additional Notes:

Day 9 Notes

DAY 10: STRATEGIC ROADMAPS, CAPSTONE PRESENTATIONS, AND PROGRAMME CLOSE

SIMULATION: Capstone Executive Panel Presentations — Enterprise Database Programming Initiative — Each participant presents their comprehensive database programming initiative proposal (10 minutes) to a simulated Central Bank executive panel comprising the facilitator and two senior peer reviewers. The proposal must cover: the specific business problem addressed, the T-SQL and Python solution architecture, security and compliance measures, performance KPIs with baseline and target values, ROI calculation, a phased deployment plan, and a 12-month technology roadmap. The panel poses challenge questions on security assumptions, KPI measurability, and change management approach. Peers score each presentation using the capstone rubric and provide written feedback.

Day 10 brings the full programme to its synthesis conclusion, where participants consolidate ten days of technical and strategic learning into enterprise-ready deployment plans and roadmaps. Capstone presentations challenge each participant to present a comprehensive database programming initiative — complete with KPIs, ROI projections, and a technology roadmap — to a panel audience simulating Central Bank leadership. The day closes with certificate recognition and a commitment to continued professional growth.

Modules: Module 10: Enterprise Deployment and Strategic Planning

Time	Module	Session / Activity	Method
09:00 - 09:30	Module 10: Enterprise Deployment and Strategic Planning	Day 10 orientation: final programme review, capstone presentation briefing, and panel assessment criteria walkthrough	Administrative
09:30 - 10:30	Module 10: Enterprise Deployment and Strategic Planning	Performance KPI definition, success metrics establishment, and ROI calculation for database programming investments	Interactive Lecture
10:30 - 10:45			Break
10:45 - 11:45	Module 10: Enterprise Deployment and Strategic Planning	Strategic roadmap development, technology evaluation, and future capability planning for evolving data requirements	Design Sprint
11:45 - 13:00	Module 10: Enterprise Deployment and Strategic Planning	Exercise: Create comprehensive deployment plan and success metrics framework for organization-wide database programming initiative	Workshop
13:00 - 14:00			Break
14:00 - 15:30	Module 10: Enterprise Deployment and Strategic Planning	★ Capstone Presentations Round 1 & 2: participants present comprehensive deployment plans and strategic roadmaps to the simulated executive panel; panel poses challenge questions on security, KPIs, and ROI assumptions	Simulation & Debrief
15:30 - 15:45			Break
15:45 - 16:30	Module 10: Enterprise Deployment and Strategic Planning	Capstone Presentations & Action Planning	Assessment
16:30 - 17:00		Closing Ceremony & Certificates	Administrative

Day 10 Learning Outcomes

1. Formulate performance KPI definitions and success metrics frameworks that quantify the ROI of database programming investments for Central Bank leadership
2. Create a strategic technology roadmap that evaluates emerging tools and maps future capability requirements to evolving organisational data needs
3. Synthesise T-SQL, Python, security, automation, and monitoring knowledge into a coherent enterprise-grade database programming initiative proposal
4. Present a comprehensive deployment plan to a simulated executive panel, demonstrating command of both technical detail and strategic business impact
5. Assess peer capstone proposals against defined criteria — technical soundness, security compliance, measurability, and stakeholder alignment
6. Demonstrate professional readiness to implement an organisation-wide database programming initiative by defending design decisions under panel questioning

FACILITATOR NOTES

- *Setup:* Prepare printed capstone assessment rubric cards for all participants to use as peer reviewers. Set up the room in a panel presentation format with a projector and a designated presenter area. Prepare certificate templates with participant names pre-filled. Allocate 12 minutes per presenter (10-minute presentation + 2-minute panel Q&A) and calculate the schedule based on cohort size.
- *Technology:* Ensure all participants can connect their laptops to the projector. Have a USB drive with all presentations backed up in case of individual laptop issues. Prepare a shared scoring spreadsheet (or printed scoring sheets) for peer assessment.
- *Groups:* Assign two peer reviewers per presenter on a rotating basis so every participant reviews at least two peers' proposals. The facilitator acts as the senior panel member and poses the most technically challenging questions. Encourage peer reviewers to ask at least one genuine question rather than only completing the rubric.
- *Energy:* Day 10 is naturally high-energy due to the culminating nature of the presentations. Manage time strictly to ensure all participants present. Use the afternoon tea break strategically between presentation rounds to maintain energy and allow presenters to reset.
- *Accessibility:* Allow participants to present in their preferred format (slides, printed document, or whiteboard sketch) as long as the required content elements are covered. Provide the assessment rubric in advance so participants know exactly what is being evaluated.
- *Prep:* Collect all participant reflective journals and action plans before the closing ceremony. Distribute post-programme support information including the 30-day check-in schedule, alumni network access details, and the 90-day impact assessment process.

Additional Notes:

Day 10 Notes

PROGRESSIVE SIMULATION

Central Bank of Lesotho — Integrated Data Pipeline and Automated Reporting Simulation

A two-phase progressive simulation set in the Central Bank of Lesotho's data operations context. Phase 1 (Day 5) requires teams to build an integrated analytical pipeline combining T-SQL extraction, Python statistical analysis, and professional visualisation to respond to an urgent Governor's office reporting request on foreign exchange reserve movements. Phase 2 (Day 6) escalates the same scenario into a fully automated monthly reporting system that must survive four compounding operational, stakeholder, ethical, and resource challenges introduced at 15-minute intervals.

Phase 1 — Day 5: Baseline Scenario

The Governor's office has issued an urgent request: a comprehensive analytical report on foreign exchange reserve movements over the past 24 months is required for a Board meeting in 90 minutes. Teams must write T-SQL queries against the training database to extract the relevant reserve and transaction data, load it into Python via pandas, apply SciPy descriptive statistics and correlation analysis to identify significant patterns, use Scikit-learn to flag anomalous reserve movements, apply time series decomposition to identify trend and seasonal components, and produce a Matplotlib/Seaborn visualisation dashboard. The final output must be a structured analytical report (Jupyter Notebook or HTML export) that a non-technical executive can interpret. Teams have 45 minutes to produce the report and 5 minutes to present findings to the group. The facilitator benchmarks each team's output against defined criteria: data accuracy (correct SQL extraction), statistical validity (appropriate test selection), visualisation clarity (chart type matches data type), and narrative coherence (findings are explained in plain language).

Team Roles: Data Extractor — responsible for writing and optimising the T-SQL extraction queries, Transformation Lead — responsible for pandas data cleaning and reshaping, Statistics and ML Lead — responsible for SciPy analysis and Scikit-learn anomaly detection, Visualisation and Narrative Lead — responsible for Matplotlib/Seaborn charts and the written report narrative

Phase 2 — Day 6: Escalation Sequence

+15 min | Operational Challenge: The scheduled Python workflow that was configured to run the monthly report has failed silently — no output was produced and no error was raised. Teams must diagnose the failure using their logging framework, identify the root cause (a simulated database connection timeout injected by the facilitator), implement a fix, and re-run the workflow successfully within the remaining simulation time.

+30 min | Stakeholder Pressure: The Deputy Governor's office contacts the team requesting that an additional chart — a month-on-month percentage change in reserve adequacy ratio — be added to the automated report before the next scheduled distribution in 20 minutes. Teams must update their Jinja2 template, add the required T-SQL calculation, regenerate the report, and confirm the updated version is queued for distribution.

+45 min | Ethical/Compliance Issue: A compliance officer flags that the automated report being distributed via email contains a column showing individual transaction counterparty names, which may constitute a data privacy breach under the Central Bank's data handling policy. Teams must identify the offending field in their T-SQL extraction query, remove or mask it, update the report template, and document the change in their audit log — all without disrupting the scheduled distribution.

+60 min | Resource Constraint: The SMTP mail server used for report distribution has become unavailable due to a network maintenance window that was not communicated in advance. Teams must implement an alternative distribution mechanism — writing the formatted report to a designated shared network folder and updating the notification system to send a plain-text alert with the file path — and verify that the fallback mechanism executes correctly within the automated workflow.

CAPSTONE PROJECT

Central Bank of Lesotho — Enterprise Database Programming Initiative Proposal

Each participant designs and presents a comprehensive enterprise database programming initiative proposal tailored to a specific operational challenge within the Central Bank of Lesotho. The proposal must integrate all ten days of learning: T-SQL query and stored procedure design, Python ETL and automation, statistical analysis and visualisation, automated reporting, API integration, performance monitoring, security and compliance hardening, and strategic deployment planning. The initiative must address a real or plausible Central Bank use case — such as automating regulatory reporting, building a foreign exchange monitoring pipeline, or implementing an automated data quality assurance system for core banking data.

Deliverable: A structured proposal document (10-15 slides or equivalent written document) covering: (1) problem statement and business case, (2) proposed T-SQL and Python solution architecture with component diagram, (3) security and compliance design including RBAC model and audit trail approach, (4) automated reporting and workflow integration design, (5) performance monitoring and KPI dashboard specification, (6) phased deployment plan with environment management approach, (7) change management and user adoption strategy, (8) success metrics with baseline values, targets, and measurement method, (9) ROI calculation with assumptions stated, and (10) 12-month technology roadmap.

The document must be presentable to the participant's line manager or IT leadership the week following the programme.

Presentation: 10 min presentation + 5 min Q&A

Assessment Rubric

Criterion	Weight	Description
Problem Statement and Business Case	10%	The business problem is clearly defined, its operational impact on the Central Bank is quantified or described with specificity, and the proposed solution is directly responsive to the stated problem.
T-SQL and Python Solution Architecture	15%	The proposed architecture correctly applies T-SQL stored procedures, Python ETL components, and integration patterns covered in the programme. Component responsibilities are clearly delineated and technically sound.
Security and Compliance Design	15%	The proposal includes a credible RBAC model, SQL injection prevention approach, Python credential management strategy, and audit trail design that addresses the Central Bank's regulatory obligations.
Automated Reporting and Workflow Integration	10%	The automated reporting pipeline design is technically feasible, includes scheduling logic, Jinja2 or equivalent templating, and a stakeholder distribution mechanism with error handling.
Performance Monitoring and KPI Dashboard	10%	The monitoring design references specific DMVs, Query Store, or Python profiling tools. KPIs are measurable, have defined baselines and targets, and alerting thresholds are justified.
Deployment Plan and Change Management	10%	The phased deployment plan is realistic, includes environment management (dev/test/prod), release procedures, and a change management approach that addresses user adoption risks.
Success Metrics and ROI Calculation	10%	Success metrics are specific and measurable. The ROI calculation includes stated assumptions, a defined measurement period, and quantified efficiency or risk-reduction benefits.
Strategic Technology Roadmap	10%	The 12-month roadmap is sequenced logically, references real tools and standards, and maps future capability requirements to the Central Bank's evolving data needs.
Presentation Clarity and Executive Communication	5%	The presentation is structured for a non-technical executive audience, uses plain language for strategic sections, and technical depth is appropriately calibrated to the panel audience.
Panel Q&A Defence	5%	The participant responds to panel challenge questions with technically accurate, well-reasoned answers and acknowledges limitations or assumptions honestly rather than overstating certainty.

POST-PROGRAM SUPPORT

30-Day Mentoring: 30-day subject matter expert pairing: each participant is matched with a Trainingcred technical mentor for two virtual 30-minute check-in sessions during the first month post-programme, focused on unblocking implementation challenges encountered when applying T-SQL and Python skills in the Central Bank environment.

Resource Library: 6-month access to the Trainingcred T-SQL and Python resource library, including annotated code repositories, additional exercise datasets, recorded walkthroughs of key programme topics, and a curated reading list of Microsoft Docs and Python documentation pages referenced during the programme.

Alumni Network: Access to the Trainingcred Data Programming Alumni Network — a moderated professional community (Microsoft Teams or equivalent) connecting programme graduates across African financial institutions for peer support, code review requests, and shared problem-solving.

Follow-up Webinar: 60-day progress review webinar: a 90-minute facilitated online session where participants share implementation progress, present one completed workplace artefact produced using programme skills, and receive group feedback from the facilitator and peers.

Impact Assessment: 90-day knowledge application and behavioural change assessment: a structured self-assessment and line-manager survey measuring the degree to which participants have applied T-SQL optimisation, Python automation, and integrated reporting skills in their Central Bank roles, with results reported to the Central Bank's training sponsor.

ALIGNED STANDARDS & FRAMEWORKS

- ISO/IEC 27001:2022 — Information security management systems; directly relevant to the database security, RBAC, and credential management content covered in Module 9
- OWASP SQL Injection Prevention Cheat Sheet (current edition) — The authoritative reference for SQL injection prevention techniques applied in Modules 3 and 9
- Microsoft SQL Server 2022 documentation — Official reference for T-SQL syntax, DMVs, Query Store, Extended Events, and execution plan interpretation covered across Modules 2, 3, and 8
- PEP 8 — Style Guide for Python Code — The governing Python coding standard applied in all Python development modules (Modules 1, 4, 5, 6, 7, and 8)
- ISO/IEC 25012:2008 — Software engineering: Data quality model; provides the framework for data source validation and quality assessment criteria applied in Module 7
- NIST Cybersecurity Framework (CSF) 2.0 — Provides the risk management and security control context for the security assessment and compliance content in Module 9
- Basel Committee on Banking Supervision — Principles for effective risk data aggregation and risk reporting (BCBS 239); directly relevant to the Central Bank's data quality, reporting automation, and audit trail requirements addressed across Modules 6, 8, and 9
- African Union — Agenda 2063 Digital Transformation Strategy; provides the continental policy context for the Central Bank's investment in data programming capability and technology modernisation addressed in Module 10

30-60-90 DAY ACTION PLAN

Timeframe	Key Milestones	Your Actions
30 Days	Within 30 days of programme completion, each participant should: (1) apply at least one T-SQL optimisation technique from Module 2 to a query currently used in production, documenting before-and-after performance metrics; (2) build and deploy one Python ETL script that automates a previously manual data	
60 Days	By 60 days post-programme, participants should: (1) have at least one automated reporting workflow running on a scheduled basis in a test or production environment, using the Jinja2 templating and scheduling patterns from Module 6; (2) have conducted a security review of at least one existing stored procedure or Python script using the assessment checklist from Module 9; (3) present a progress update at the 60-day group webinar, sharing one completed workplace artefact; and (4) have drafted the first version of their capstone deployment plan for review by their line manager.	

90 Days	By 90 days post-programme, participants should: (1) have presented their database programming initiative proposal to their line manager or IT leadership team, using the capstone deliverable as the basis; (2) have established at least one performance monitoring dashboard tracking database or Python workflow metrics using the DMV and logging patterns from Module 8; (3) complete the 90-day impact assessment survey and line-manager evaluation; and (4) have identified the next capacity development step — whether advanced T-SQL, Python data engineering, or cloud database platforms — and enrolled in	
----------------	---	--

PROGRAM NOTES

or planned the relevant next learning activity.

Use this space for additional notes, observations, and action items.

